

Identifying System-on-Chip Security Assets with Structure-Based Analysis

Wei-Kai Liu[§], Benjamin Tan[†], and Krishnendu Chakrabarty[‡]

[§]Duke University, [†]University of Calgary, [‡]Arizona State University
weikai.liu@duke.edu, benjamin.tan1@ucalgary.ca, krishnendu.chakrabarty@asu.edu

Abstract—In the evolving field of hardware design, ensuring the security of System-on-Chips (SoCs) has become increasingly vital. As SoCs grow in complexity, integrating components from various sources, the identification and protection of security assets are crucial to prevent vulnerabilities. Traditional methods of identifying these assets are manual and time-intensive. To address this challenge, automated tools for security asset identification are essential, enabling faster and more accurate detection of critical assets early in the design process. In this paper, we propose a framework for the automated identification of security assets within SoCs. By transforming register-transfer level (RTL) code into graphs and leveraging deep neural networks (DNNs) to classify assets based on their structural patterns, our approach can effectively differentiate between security and non-security assets. Experimental results show that the proposed method achieves high classification accuracy, with the model reaching up to 99% accuracy in identifying security assets, significantly reducing the need for manual intervention.

I. INTRODUCTION

System-on-chips (SoCs) have become vital to modern technology, driving innovation in industries ranging from consumer electronics to healthcare, mobile devices, and autonomous vehicles [1]–[4]. As these systems increasingly manage sensitive data and perform critical operations, ensuring their security has become a top priority for designers and system integrators [5]–[8]. Key security elements within SoCs, such as encryption keys, configuration data, and access control signals, must be rigorously protected to maintain the system’s integrity and confidentiality [9].

As SoCs become more complex, integrating a diverse array of IPs from various sources, the potential for security vulnerabilities increases [10]–[13]. Furthermore, the interactions between these different components can lead to unforeseen security issues that malicious users could exploit after deployment. Common Weakness Enumerations (CWEs) proposed by MITRE highlight various examples of prevalent security bugs and the corresponding exploits observed in the industry [14]. While security assets are typically integrated with various protection mechanisms to ensure the safety of each IP, these mechanisms can be improperly implemented, leaving assets vulnerable to exploitation [15]. As highlighted by [14], many security assets are inadequately protected due to incorrect access control policies or misconfigurations. Although some hardware patching techniques have been proposed to protect

security assets during runtime [5], [16], they cannot guarantee complete protection. Therefore, system integrators must thoroughly review all security assets across IPs and their associated protection mechanisms to ensure the SoC’s security.

To address these challenges, it is crucial to prioritize security from the earliest stages of SoC design [17]. This involves identifying the most critical assets that require protection and ensuring their security throughout the design process. Several approaches have been proposed to safeguard assets in SoCs, such as incorporating security policies or properties alongside simulations to verify the safety of designs [18]–[21]. However, these methods often require the manual definition and inspection of security assets and their corresponding protection mechanisms, relying heavily on the designer’s efforts. Previous works [22], [23] identify assets in a design by tracing the data flow from a list of primary assets provided by IP vendors. However, this approach relies heavily on designers, so it may overlook other security assets if the primary assets are not explicitly defined.

To enable pre-silicon security analysis, we propose a method for identifying security assets in SoCs at the register-transfer level (RTL) automatically. Therefore, system integrators can easily gather security assets of different IPs without manually delving into each design. Our approach captures the data flow and control flow of each signal, converting them into numerical vectors that serve as signatures for identifying other assets with similar structures. Since assets of the same type usually have the same protection mechanism, i.e., the data/control flow is similar, this approach can effectively identify security assets by clustering signals with similar structures. By minimizing the manual workload and reducing the amount of user-provided information, this method aims to streamline the early stages of security analysis, enhancing both its robustness and accuracy.

Our main contributions are as follows:

- We introduced a framework to automate the identification of security assets in SoCs by analyzing structural patterns of RTL signals, reducing the need for human efforts.
- We developed a structure-based approach that captures both control flow and data flow information from RTL, facilitating the identification of security assets through graph analysis.
- We employed a DNN model to classify security and non-security assets and achieve high accuracy across multiple SoC platforms.

The remainder of the paper is organized as follows: Section II provides an overview of asset definitions and security counter-

*This work was supported in part by the Semiconductor Research Corporation (SRC) under Tasks 3065.001, 3106.001, and 3198.001, and by a gift from Intel Corporation.

measures, while Section III reviews relevant prior work. The proposed framework is introduced in Section IV, followed by experimental results in Section V. Finally, the paper concludes in Section VI.

II. BACKGROUND

A. Definition of Security Assets

Accellera’s SA-EDI standard [24] defines assets as “anything of value or importance that is used, produced, or protected within the IP.” OpenTitan, the largest commercial-scale open-source hardware root-of-trust integrating various security IPs, has documented all asset types in their projects, with a selection shown in Table I. However, for SoC security, we focus only on assets that impact information security, specifically those that affect confidentiality, integrity, and availability, as represented by the CIA triad. These are referred to as security assets.

In previous works [18], [23], security assets are primarily classified into two: primary assets and secondary assets.

- **Primary Assets:** Critical design components essential to the security of a SoC. Their integrity is vital, and their compromise can lead to significant security breaches.
- **Secondary Assets:** Supportive components linked to primary assets. While not directly critical, their compromise can weaken the overall security of the SoC.

For example, keys or passwords are classified as primary assets, while data registers used to transmit primary assets are considered secondary assets. Although data registers may not typically contain sensitive information, they can still be exploited by attackers to access primary assets if left unprotected. Similarly, as shown in Table I, some asset types, such as RST and CTRL, are not usually security-relevant but can become secondary assets in specific scenarios. As such, there is no definitive standard for classifying security assets. Therefore, this paper proposes an approach that evaluates each signal’s data and control flow complexity, as well as its relationship with other signals, to identify security assets, rather than relying solely on asset type classifications.

TABLE I: Common asset types proposed by [25].

Asset Name	Meaning
KEY	A key (secret data)
CTRL	Logic used to steer hardware behavior
ADDR	An address
DATA_REG	A configuration data register
CTR	A counter
RST	A reset signal
MEM	A generic data memory
CONFIG	A configuration register
BUS	Data transferred on a bus
CONSTANTS	A netlist constant

B. Security Countermeasures

Security countermeasures are protection mechanisms designed to safeguard security assets. For security-critical IPs,

these countermeasures are commonly implemented to prevent attacks or side-channel analysis. The specific countermeasures employed vary depending on the type of asset. Some common security countermeasures used by OpenTitan are listed in Table II.

Since assets of the same type typically share the same countermeasures, especially when they originate from the same design house, these countermeasures can be leveraged for identifying security assets. Such countermeasures introduce additional data or control flows to the assets to provide protection. For example, **REGWEN** and **REGREN** condition asset access on the *wen* and *ren* signals, ensuring that assets can only be accessed when these enable signals are pulled high. Usually, all the *wen* and *ren* signals are typically generated in similar ways or through modularized logic, with differences only in their switching conditions. Thus, if one security countermeasure can be identified automatically, all assets using the same protection mechanism can be identified accordingly. Since the number of security countermeasures is typically much smaller than the number of security assets, designers can significantly reduce effort by providing a list of countermeasures instead of security assets.

TABLE II: Common countermeasures used in OpenTitan [25].

Countermeasure Name	Meaning	Associated Assets
REGWEN	A register write enable is used to protect the asset from write access	ADDR, CTRL
SHADOW	The asset has a shadow replica to cross-check against	CONFIG
INTEGRITY	The asset has integrity protection from a computed value	CONFIG, MEM
REGREN	A register read enable is used to protect the asset from read access	CONFIG, MEM

C. Hyperflow Graph

A hyperflow graph (HFG) is a specialized intermediate representation designed to support and enhance security analysis and visualization in hardware systems [26]. It maps out the intricate connections between different design variables, capturing both direct and conditional data flows. This graph not only helps in tracing how information moves through the system but also provides crucial insights into security-relevant aspects such as key assets, boundary definitions, and the execution of specific functions during simulation. In the proposed work, we simplified the original HFG proposed in [26] to represent the relationship among signals.

In this HFG graph, each node represents a signal, or a single bit of a signal if each bit has different control or data flows. An edge exists between two nodes whenever there are any control or data flows between them. A solid edge represents a direct assignment between two signals, while a dotted edge is used for a conditional assignment. For example, in the following RTL statement:

```

if (ctrlA or ctrlB)
    dataB <= dataA;
else
    dataB <= dataC;

```

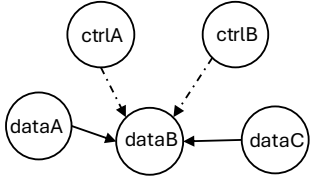


Fig. 1: An example hyperflow graph.

two solid edges pointing to *dataB* from *dataA* and *dataC* and two dotted edges pointing to *dataB* from *ctrlA* and *ctrlB* will be created.

An example hyperflow graph is shown in Figure 1. Since the value of *dataB* is determined by either *dataA* or *dataC*, these direct assignments are represented by solid edges. The signals *ctrlA* and *ctrlB*, which control the value of *dataB*, are connected with dotted edges to indicate control flow. In simpler terms, signals on the right-hand side (RHS) will have solid edges pointing to the signals on the left-hand side (LHS), while signals involved in logical statements, such as if and case statements, will be connected with dotted edges to the signals on the LHS within their respective scopes.

The HFG effectively clarifies the relationships between signals by stripping away unnecessary information from the RTL and focusing solely on the data and control flows among signals. By tracing the nodes through different types of edges—solid for direct assignments and dotted for control flows—we can identify distinct structural patterns associated with various assets, as each asset type exhibits unique characteristics.

III. RELATED PRIOR WORK

Several approaches have been proposed to protect security assets in SoCs, but most rely heavily on manual identification or predefined asset sets. For example, Meza et al. [26] used information flow tracking with hyperflow graphs to verify the security of the OpenTitan hardware root of trust, focusing on OpenTitan’s security assets. However, their method required manual inspection and did not automate asset identification.

Dipu et al. [27] introduced a framework to generate system architectures and comprehensive security policies tailored to security assets, considering various vulnerabilities. While their approach is policy-driven, it requires the security assets to be defined in advance. Similarly, design-for-test (DFT) logic, which enhances the observability and controllability of security assets for testing purposes, must be reviewed carefully. As noted in [28], if not properly secured, DFT logic can be exploited by attackers to gain access to these assets. Their study also relied on manually selecting security assets for analysis.

[9] proposed an automated approach for scanning SoC designs to identify potential security vulnerabilities based on the CWE database [14]. However, their method lacks generalizability, as it only implements algorithms for a limited set of specific weaknesses. Additionally, each scanner is tailored to a particular CWE, which means that significant designer

effort would be required to develop a comprehensive set of algorithms capable of discovering a broader range of security bugs in SoCs.

[18] proposed an algorithm to generate security-critical properties for SoCs by identifying vulnerabilities and creating corresponding protective properties for selected assets. Ray et al. [20] and Hu et al. [19] evaluate the security properties of hardware components systematically, identifying potential vulnerabilities, and generating policies to protect IP assets during verification. Despite these advancements, all of these methods still depend on manual asset identification or predefined asset sets, limiting their scalability and automation potential.

[23] is the only work we found that identifies security assets automatically. By defining a set of primary assets, it employs algorithms to track information flow and subsequently identify secondary assets. However, this approach is limited to identifying only those secondary assets that have data or control flow to or from the predefined primary assets, which must be provided in advance. As a result, if IP vendors fail to list all the primary assets, some security assets may be overlooked. To address this limitation, the proposed work aims to automatically identify both primary and secondary assets without requiring the set of primary assets from designers.

IV. PROPOSED FRAMEWORK

The proposed framework is shown in Figure 2. The process begins by collecting the source code and design packages from a given SoC. We then parse the Verilog/SystemVerilog design code and generate abstract syntax trees (ASTs) using Verific’s parser [29]. Each AST corresponds to a module, capturing all the essential information from the RTL code. To extract the necessary details for building the HFG, we implement various visitors to traverse these ASTs.

A. Building HFG

To construct the HFG, we primarily need to capture the control flow and data flow between signals. To achieve this, we design visitors that detect all conditional blocks, such as “case” and “if” statements, as well as conditional assignments, where control signals influencing the values of other signals are identified. Any signals appearing in the sensitivity list are recognized as control signals, regardless of the boolean operators used between them. In addition, both blocking and non-blocking assignments are identified, and the signals on the LHS and RHS are mapped to establish the corresponding data flow. Similarly, when multiple signals appear on the RHS (e.g., $C = A \& B$), the data flow is noted as originating from both *A* and *B* towards *C*. To trace signals across hierarchical modules, we record the port names of all submodules and their corresponding signal names. Once all modules have been visited, a bottom-up traversal is performed to fill in any missing connections where signals pass through submodules. During this traversal, missing submodules’ subgraphs are created and linked to the corresponding module. We implement a tool in python to plot the HFG to ensure the correctness of our implementation. A visualization of the HFG for *opt_ctrl.sv* is

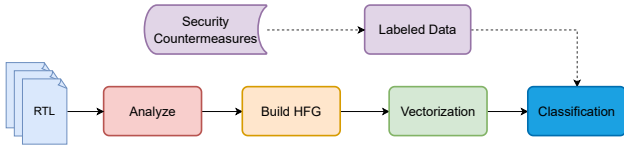


Fig. 2: The proposed framework for security asset identification.

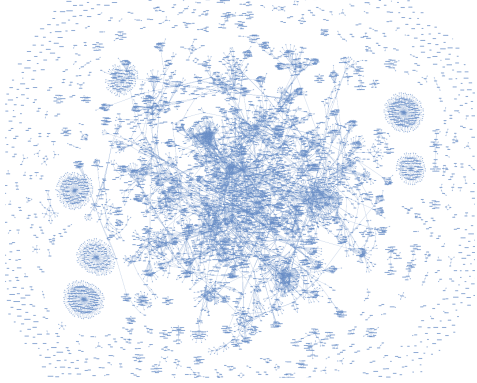


Fig. 3: HFG visualization of the OTP controller in OpenTitan. A multi-bit signal (e.g., a bus) is represented as either a single node or multiple nodes, where each node corresponds to a bit with distinct data or control flows. When output signals in the top module lack data/control flows into submodules, cluster-type sub-graphs are generated.

presented in Figure 3, allowing for easy tracing of signals’ data/control flows and providing a clear understanding of the relationships among signals.

B. Vectorization

To perform security asset identification, we extract the structural information from the HFG of each signal to represent its connections. Since the HFG is a directed graph with two types of connections, representing data and control flows respectively, each node can have up to four distinct types of neighbors: conditional predecessors (CP), direct predecessors (DP), conditional successors (CS), and direct successors (DS). Predecessors are nodes that have edges pointing to the target node, while successors are nodes that the target node points to. Direct connections represent data flow, and conditional connections represent control flow, visualized as solid and dotted lines, respectively, as shown in Figure 1.

The number of these four different types of neighbors provides insight into a signal’s characteristics. For example, reset, enable, or other control signals that frequently appear in ‘if’ or ‘case’ statements will have many conditional successors and few, if any, direct successors. Conversely, data registers or signals carrying memory values, which are often accessed or passed with other values, will have many direct successors and predecessors. By analyzing each signal based on these characteristics, we can effectively categorize signals into distinct groups. These four numbers can then be combined into a vector, serving as a unique signature for each signal.

However, some signals may appear similar based on these four numbers but actually have significantly different connections beyond the first layer of neighbors. To address this issue, we incorporate additional information from the HFG by capturing two layers of neighbors for each signal. In implementation, we achieve this by combining the vectors of all neighboring nodes of the target node. To prevent information loss during the combination process, we group neighbors by type (CP, DP, CS, DS) and calculate the average vector for each group. Therefore, each vector can represent the general characteristics for each type of neighbors of the target node, while the original 4-dimensional vector still specifies the number of each type of neighbors. As a result, in addition to the original 4-dimensional vector, we generate four additional 4-dimensional vectors corresponding to each type of connection for the target node. Ultimately, this results in a 20-dimensional vector representing a signal, which is then used for classification.

C. Classification

Once each signal is vectorized, these vectors serve as unique signatures for the signals, allowing for classification. We employ a Deep Neural Network (DNN) to train a model capable of identifying security assets. To train the model, labeled datasets are required, which must be provided by the designers. However, designers only need to supply a small dataset representing each type of security asset, as different asset types exhibit distinct security countermeasures. These countermeasures result in varied connections within the HFG, and the model is trained to recognize and classify these differences effectively.

By identifying the structure of each protection mechanism, this approach is agnostic to the varying keywords or naming conventions used by different IP vendors. Additionally, because we consider not only the closest neighbors of the target node (i.e., those with a distance of 1), the method is more robust against small design modifications. For example, even if two assets are protected using the same mechanism, the number of CP, DP, CS, and DS connections may differ slightly. In such cases, the DNN can still classify them into the same category, as long as the Euclidean distance between the signals is within a threshold and their signature vectors exhibit similar characteristics.

Although many Graph Neural Network (GNN) approaches have demonstrated strong performance [30], [31], graph transferability remains a significant challenge [32]. Graph transferability refers to the problem where a GNN model, trained on one graph, struggles to generalize to another, resulting in poor performance on unseen graphs. However, our objective is to identify specific structural patterns within security assets. Therefore, regardless of how the HFG is constructed, the known assets must still be accurately identified. Since HFGs can vary significantly between different SoCs or across different hierarchical levels within the same SoC, we chose not to use GNNs for classification. Moreover, as each dimension of the signal vector contains all the necessary information, a

DNN is much more energy- and time-efficient compared to a GNN.

V. EXPERIMENTAL RESULTS

A. Experimental Setup and DNN Hyperparameters

Our experiments are conducted on a machine equipped with an NVIDIA TITAN V. The training hyperparameters are as follows: the model is trained for 100 epochs using the Adam optimizer with a learning rate of 10^{-3} , which decayed by a factor of 0.1 every 5 epochs. A batch size of 32 was used during training. The DNN model consists of four fully connected layers, each followed by batch normalization and ReLU activation. A dropout rate of 0.3 was applied after each fully connected layer to reduce overfitting by randomly disabling neurons during training. The number of neurons in the layers are 128, 64, 16, and a task-specific output size. Input normalization was applied to ensure stable and efficient training. The cross-entropy loss function was employed for classification tasks, with softmax applied at the output layer. We utilized 10-fold cross-validation to ensure model robustness and generalization. The primary evaluation metric used was accuracy, measured on the validation set during each fold.

B. Overview of SoCs

We conduct the experiments on OpenTitan and OpenPiton (from Hack@Dac21 [33]). The number of scanned modules by Verific, along with the number of nodes and edges in the HFGs, and other information of these SoCs, are presented in Table III. Due to the absence of labeled data from these SoCs, we manually reviewed and selected security assets based on our understanding of the system. OpenTitan, designed with security as a primary focus, has more structured countermeasures, security features, and clearer asset identifiers. A block diagram of the OpenTitan Earl Grey chip is shown in Figure 5, showcasing its multiple security peripherals within the system.

For OpenTitan, we selected assets in alignment with the proposed countermeasures, as shown in Table II. We identified security assets by searching for specific keywords associated with the countermeasures, including “regwen”, “gated_we”, “shadowed”, and “intg_err”. In contrast, OpenPiton is more general-purpose, requiring more generalized keywords for asset classification. For OpenPiton, we use “mem”, “en”, “_ctrl”, “key” to search for security assets, while keywords such as “rdata”, “wdata”, and “address” to search for assets that may not be security-relevant. Signals that did not match any of the specified keywords were classified as non-security assets.

The runtime for completing the entire framework is presented in Table III. In contrast to manual inspection—which typically takes several minutes to review an RTL file with 100 lines, and requires a deep understanding of each IP to accurately identify security assets—our automated approach significantly reduces the time required. Specifically, our method identifies security assets from over 300 modules, comprising nearly 240k lines of code, in just 13 seconds.

TABLE III: Overview of SoC metrics used in the experiment, including the number of scanned modules, lines of code parsed, hyperflow graph details, and the corresponding runtime.

SoC Name	# of Modules	LoC	# of Nodes	# of Edges	Time (s)
OpenTitan	332	239790	1455692	2375044	13.6
Hack@DAC21 (OpenPiton)	128	166396	1397004	3076387	11.5

C. Results Analysis

In OpenTitan, we used four different keywords to classify signals into distinct asset categories, i.e., “regwen”, “gated_we”, “shadowed”, and “intg_err”. Initially, we collected the signals associated with each keyword and ran the classification model. The resulting classification achieved an accuracy of 99%, with the confusion matrix depicted in Figure 4 (a). This high accuracy demonstrates the model’s ability to distinguish between the selected asset types when only security-relevant signals are present.

Next, we incorporated non-security assets into the dataset. As shown in Figure 4 (b), the classification accuracy dropped to 92%. A closer look at the confusion matrix reveals that many non-security assets (label 4) were incorrectly classified as different types of security assets. This misclassification is likely due to the limitations of our keyword-based labeling system. While the keywords effectively capture many security assets, there are other security-relevant signals that do not match these keywords but share similar protection mechanisms. As a result, the model misclassifies some non-security assets that exhibit similar patterns.

Lastly, we combined the four asset types into a single category, labeling all of them as “security assets,” and performed a binary classification against the non-security assets. The result, as shown in Figure 4 (c), achieved a 94% accuracy. This binary classification simplified the task, but still highlights the challenges of accurately identifying security signals.

We then performed the same three classification tasks for OpenPiton. The results yielded accuracies of 93%, 86%, and 88%, following a similar trend as observed with OpenTitan. When incorporating different classes of security assets with non-security assets, the accuracy is the lowest for both SoCs. This is likely due to some security assets being mislabeled as non-security assets during the keyword-searching process, which, although effective, is not perfect.

However, when the various categories of security assets are grouped together for classification, we achieve the highest accuracy. This indicates that the different types of security assets exhibit distinct structural patterns in their vector representations, allowing for easy differentiation. Notably, these assets come from different IPs, which means their connections may not be identical, yet they are still correctly classified under the same category. Even non-security relevant assets in OpenPiton, such as “rdata”, “wdata”, and “address”, can be accurately identified. This demonstrates that the DNN model is effective at distinguishing signals based on their structural patterns, as long as these patterns differ sufficiently.

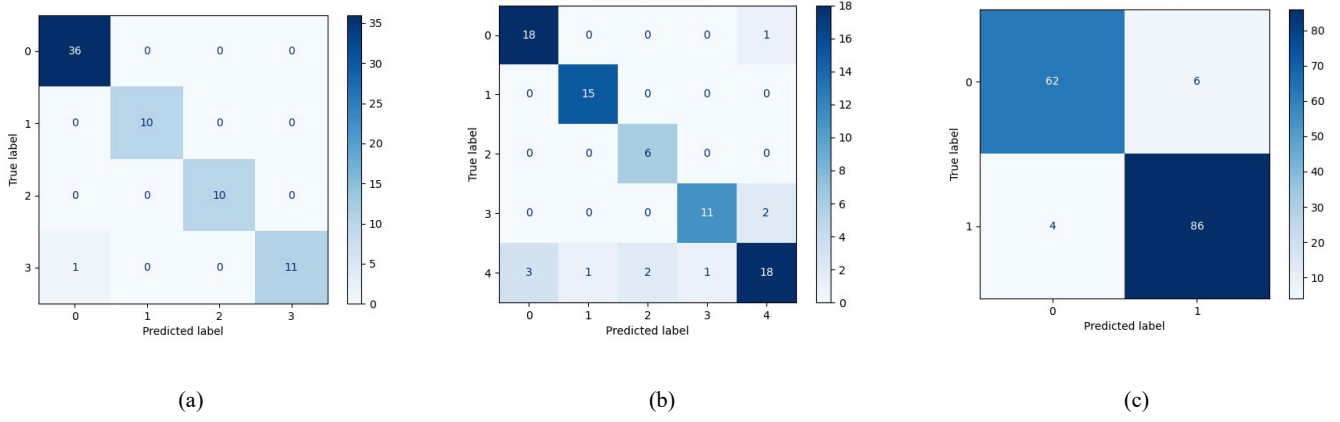


Fig. 4: Confusion matrices for security asset classification. (a) shows the classification results for four distinct types of security assets in OpenTitan, based on the keywords: regwen, gated_we, shadowed, and intg_err. (b) presents the classification results for both the four security asset types (labels 0-3) and non-security assets (label 4). (c) displays the binary classification results, differentiating between security assets (label 0, matching the four keywords) and non-security assets (label 1, signals not matching any keyword).

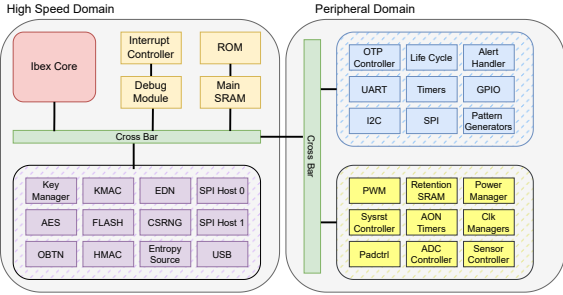


Fig. 5: Block Diagram of the OpenTitan Earl Grey Chip [25].

To further understand the importance of each input vector dimension, we plotted the absolute values of the first layer's model weights, as shown in Figure 6. The 20 dimensions consist of the following:

- 4 dimensions represent the number of each type of neighbor (CP, DP, CS, DS) for the target node.
- 16 dimensions are the averaged characteristics of neighbors in each of the four groups (CP, DP, CS, DS), with 4 dimensions per group (e.g., CP-CP, CP-DP, ..., DS-DS), representing the interactions between different neighbor types.

The weight distribution appears relatively uniform, suggesting that each dimension of the signature vector contributes similarly to the classification decision. This uniform distribution highlights that the DNN is leveraging all available features without relying heavily on any single dimension, which indicates a balanced representation of the input data.

Such uniformity in the weight distribution also suggests that the extracted features are comprehensive and non-redundant, meaning that no single feature dominates the decision-making process. This is particularly important as security assets often

display subtle variations that require a broad set of features for accurate identification. This distribution implies that the DNN is learning to generalize well across various protection mechanisms, using multiple dimensions to capture different aspects of the signals.

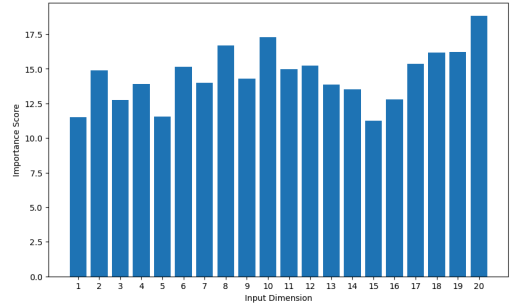


Fig. 6: The importance of each input dimension based on first layer weight.

VI. CONCLUSION

In this paper, we introduced a framework for the automated identification of security assets in SoCs. By constructing hyperflow graphs (HFGs) from RTL code and employing deep neural networks (DNNs) for classification, we were able to accurately identify security-relevant signals. Our experiments showed that the model effectively distinguished between security and non-security assets, even in the presence of architectural complexities and limitations associated with keyword-based labeling. By leveraging structural patterns within the signals, the proposed method significantly reduces manual intervention, streamlining and improving the efficiency of the security analysis process. The model achieved classification accuracies of 99% and 93% across different asset types for two distinct SoCs, demonstrating its robustness and adaptability.

REFERENCES

- [1] K. Tsantikidou and N. Sklavos, "Flexible security and privacy, system architecture for IoT, in healthcare," in *IFIP/IEEE International Conference on Very Large Scale Integration (VLSI-SoC)*. IEEE, 2022.
- [2] A. Ishtiaq, M. U. Khan, S. Ali, K. Habib, S. Samer, and E. Hafeez, "A review of system on chip (SoC) applications in internet of things (IoT) and medical," in *International Conference on Advances in Mechanical Engineering*, 2021.
- [3] D. Ariando, C. Chen, M. Greer, and S. Mandal, "An autonomous, highly portable NMR spectrometer based on a low-cost System-on-Chip (SoC)," *Journal of Magnetic Resonance*, 2019.
- [4] S. R. Sridhara *et al.*, "Microwatt embedded processor platform for medical system-on-chip applications," *IEEE Journal of Solid-State Circuits*, 2011.
- [5] W.-K. Liu, B. Tan, J. M. Fung, R. Karri, and K. Chakrabarty, "Hardware-supported patching of security bugs in hardware IP blocks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
- [6] J. Abella *et al.*, "Security, reliability and test aspects of the risc-v ecosystem," in *IEEE European Test Symposium (ETS)*, 2021.
- [7] F. Farahmandi, A. Srivastava, G. Di Natale, and M. Tehranipoor, "Introduction to the special issue on CAD for security: Pre-silicon security sign-off solutions through design cycle," 2023.
- [8] M. Akyash and H. M. Kamali, "Evolutionary large language models for hardware security: A comparative survey," in *Proceedings of the Great Lakes Symposium on VLSI*, 2024.
- [9] B. Ahmad *et al.*, "Don't sweat it: Toward cwe analysis techniques in early stages of hardware design," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2022.
- [10] B. Ahmed *et al.*, "Quantifiable assurance: from IPs to platforms," *arXiv preprint arXiv:2204.07909*, 2022.
- [11] J. Knechtel *et al.*, "Towards secure composition of integrated circuits and electronic systems: On the role of EDA," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2020.
- [12] Z. Wang, L. Alrahis, L. Mankali, J. Knechtel, and O. Sinanoglu, "Llms and the future of chip design: Unveiling security risks and building trust," *arXiv preprint arXiv:2405.07061*, 2024.
- [13] H. Pearce, R. Karri, and B. Tan, "High-level approaches to hardware security: A tutorial," *ACM Transactions on Embedded Computing Systems*, 2023.
- [14] S. Christey, J. Kenderdine, J. Mazella, and B. Miles, "Common weakness enumeration," *Mitre Corporation*, 2013.
- [15] G. Dessouky, D. Gens, P. Haney, G. Persyn, A. Kanuparthi, H. Khattri, J. M. Fung, A.-R. Sadeghi, and J. Rajendran, "Hardfails: insights into software-exploitable hardware bugs," in *USENIX Security Symposium*, 2019.
- [16] W.-K. Liu, B. Tan, J. M. Fung, and K. Chakrabarty, "Theoretical patchability quantification for IP-level hardware patching designs," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.
- [17] S. R. Rajendran, N. F. Dipu, S. Tarek, H. M. Kamali, F. Farahmandi, and M. Tehranipoor, "Exploring the abyss? unveiling systems-on-chip hardware vulnerabilities beneath software," *IEEE Transactions on Information Forensics and Security*, 2024.
- [18] N. Farzana, F. Rahman, M. Tehranipoor, and F. Farahmandi, "SoC security verification using property checking," in *IEEE International Test Conference (ITC)*, 2019.
- [19] W. Hu, A. Althoff, A. Ardeshiricham, and R. Kastner, "Towards property driven hardware security," in *International Workshop on Microprocessor and SOC Test and Verification (MTV)*, 2016.
- [20] S. Ray and Y. Jin, "Security policy enforcement in modern SoC designs," in *International Conference on Computer-Aided Design (ICCAD)*, 2015.
- [21] S. Ray, E. Peeters, M. M. Tehranipoor, and S. Bhunia, "System-on-chip security assurance: Architecture and validation," *Proceedings of the IEEE*, 2017.
- [22] N. Farzana, A. Ayalasomayajula, F. Rahman, F. Farahmandi, and M. Tehranipoor, "Saif: Automated asset identification for security verification at the register transfer level," in *IEEE VLSI Test Symposium (VTS)*, 2021.
- [23] A. Ayalasomayajula, N. Farzana, M. Tehranipoor, and F. Farahmandi, "Automatic asset identification for assertion-based SoC security verification," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [24] Accellera Systems Initiative, "Security annotation for electronic design integration standard," 2021.
- [25] "Opentitan," <https://opentitan.org/>.
- [26] A. Meza and R. Kastner, "Information flow coverage metrics for hardware security verification," *arXiv preprint arXiv:2304.08263*, 2023.
- [27] N. F. Dipu, F. Farahmandi, and M. Tehranipoor, "SoC security properties and rules," *Cryptology ePrint Archive*, 2021.
- [28] G. K. Contreras, A. Nahiyan, S. Bhunia, D. Forte, and M. Tehranipoor, "Security vulnerability analysis of design-for-test exploits for asset protection in SoCs," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2017.
- [29] "Verific design automation," <http://www.verific.com>.
- [30] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, and M. Sun, "Graph neural networks: A review of methods and applications," *AI open*, 2020.
- [31] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, 2020.
- [32] L. Ruiz, F. Gama, and A. Ribeiro, "Graph neural networks: Architectures, stability, and transferability," *Proceedings of the IEEE*, 2021.
- [33] HACK@EVENT, "HACK@DAC21," 2022. [Online]. Available: <https://hackatevent.org/hackdac21/>